Interrupted Adaptation in Learned World Models: An Empirical Study of Restart Policies, Window Length and Experience Replay

Aditya Keshri

Code, data and logs: https://github.com/Optimus2007/world-model-autonomous-agent

Abstract

An agent that plans with a learned dynamics model must detect changes in the true dynamics and correct its model. A common pipeline opens an adaptation window when a change detector raises an alarm, fine tunes the model on transitions collected in the window, and ignores further alarms until the window closes. We study what happens when the dynamics change again while such a window is open. In a planar point mass with unannounced actuator rotations, controlled by cross entropy planning over a neural network ensemble, we compare three interruption policies (ignore, restart the window on a stricter re-detection, and ignore with a shorter window), with and without replay, on six change schedules and ten paired evaluation seeds. Restart improved return after every rotation second change, in 40 of 40 paired comparisons without replay and 39 of 40 with replay, with no measurable cost when only one change occurred. The shorter window, however, recovered at least half of that improvement in every rotation schedule and all of it in two of four, so much of the benefit comes from ending a stale window sooner. Replay helped far more under ignore than under restart, largely by compensating for stale windows, and it reduced return in every seed after a single permanent change and after a gain change. Restart is a single model case of existing context detection methods; our contribution is a controlled measurement in one small simulated system, whose effect sizes may not transfer to other settings.

1 Introduction

A model-based agent is only as good as the agreement between its model and the world. When the dynamics change without warning, as with a miscalibrated actuator, a picked up payload, or a partial motor fault, plans computed under the old model can fail badly. A simple and widely used remedy is to watch the one step prediction error, raise an alarm when it rises persistently, and fine tune the model on transitions gathered after the alarm. Because a single transition is not enough to fine tune on, the agent opens an *adaptation window*: it collects data, fine tunes after a minimum number of transitions, fine tunes again as more arrive, and closes the window after a fixed budget. While the window is open, further alarms are usually ignored, since the model is known to be wrong and retraining on every alarm would never end.

This design has a blind spot. If the dynamics change again while the window is open, the agent keeps fitting one model to a mixture of transitions from two regimes, and it cannot react until the window closes. Recurring or quickly repaired faults make this more likely: a system can leave a regime and return to it within the lifetime of one window. In earlier stages of the project behind this paper we observed exactly this failure. An agent that had adapted well to a 60° actuator rotation needed several episodes to recover when the rotation reverted during the adaptation window.

The obvious fix is to let the agent restart its window when a new change is detected inside it. We implemented this as a small state machine with a second, stricter detector that is armed only after the window's first fine tune. Restart is not a new idea. Discarding stale data on detected change and refusing to trust a detector until enough recent data exist are standard in stream learning [3, 7], and model-based context detection methods such as MBCD [2] already route post change data to a new

model after a warm-up period. What is less clear is *why* restart helps in a window based pipeline, and how much it matters once other design choices are taken into account. A restart does two things at once: it reacts to the new alarm, and it ends the old window early. A shorter window would also end the old window early, without any second detector. Replay of old data, the usual defence against forgetting, also changes how much a stale window hurts.

This paper asks: *how should a single learned world model with window based fine tuning handle a new dynamics change that occurs before adaptation to an earlier change has finished, and how does replay affect performance when earlier dynamics return?* We break this into four research questions.

RQ1 Does restarting the window on re-detection improve control after a second change, compared with ignoring the alarm?

RQ2 What does restart cost in false restarts, compute, and forgetting of earlier regimes?

RQ3 When does a fixed replay ratio improve whole run return, and does the answer depend on the interruption policy?

RQ4 Is one replay ratio adequate for both permanent and reversible changes?

Alongside RQ1 we test a competing explanation (C1): restart helps only because it ends the stale window sooner. The test is a shorter window under the original ignore policy.

Our contributions are empirical. (i) A seed paired measurement of the cost of suppressing alarms during an adaptation window, over six change schedules, three of which were never used for tuning. (ii) A direct test of the window length explanation, which turns out to account for much of restart's benefit. (iii) A characterisation of how replay interacts with the interruption policy, including cases where replay hurts. All results come from one small simulated system, and we make no claim that the effect sizes generalise.

2 Related work

Model-based control under changing dynamics. Our planner follows the trajectory sampling scheme of PETS [4], which plans with cross entropy optimisation over an ensemble of learned dynamics models; our ensemble members are deterministic [11]. Several methods avoid discrete change handling altogether. Nagabandi et al. [15] meta learn a dynamics prior and adapt it continuously from a recent window of transitions, and context conditioned models [12] infer a latent context from recent data. In these designs there is no adaptation window to interrupt. Detect then adapt pipelines are also common in deep reinforcement learning. Padakandla et al. [17] couple change point detection on environment statistics with a learning algorithm, and Liu et al. [13] monitor the joint distribution of states and actions. LLCD [14] runs Bayesian online change point detection [1] on the latent states of a learned world model and adds a temporary regulariser to model training after a detected change. That temporary phase plays the role of our adaptation window. We read LLCD in the first author's open thesis rather than the journal version.

Recurring contexts and model libraries. The closest prior work keeps several models. RL-CD [5] tracks the prediction quality of each model and creates a new one when none fits; we know it through the description by Alegre et al. [2]. MOLe [16] assigns each time step to a component of a mixture of dynamics models with a Chinese restaurant process prior and recalls old components when earlier tasks return. Xu et al. [22] do the same with a Dirichlet process mixture of Gaussian process experts. MBCD [2] runs a multivariate CUSUM [18] of log likelihood ratios against every known context model and a new context alternative, and reuses an earlier model when its statistic wins. Two features of MBCD matter here. Detection statistics are computed only after a warm-up period in each new context, which the authors argue is a necessary assumption, and post change transitions go to the new context's dataset rather than mixing with the old one. Our restart policy is essentially the single model special case of this design. What our agent lacks, and MBCD, MOLe and

the Gaussian process mixture have, is a library of models that can be reused when a context recurs. In those methods a change during adaptation is handled structurally, because each step is assigned to a context.

Stale data and forgetting. Rahimi-Kalahroudi et al. [19] show, in the LoCA setting [21], that a first in first out replay buffer retains stale transitions that corrupt a deep world model after a local change, and they propose deleting only old samples in the neighbourhood of new ones. This is the closest statement of our stale data problem, though for local reward changes rather than global dynamics changes. Experience replay is the standard defence against catastrophic forgetting in continual reinforcement learning [20], alongside regularisation such as EWC [10] and task conditional hypernetworks [8]. DRAGO [6] retains world model knowledge across tasks through generative rehearsal, under known task switches and shared dynamics, which is complementary to our setting. Khetarpal et al. [9] survey continual reinforcement learning more broadly.

Stream learning. ADWIN [3] grows its window while the data look stationary and shrinks it on detected change, discarding stale data. DDM [7] monitors the online error rate with warning and drift levels, and common implementations do not signal a change before a minimum number of samples. Restart combines both moves in a window based fine tuning pipeline.

What this study adds. None of the sources we read reports a controlled measurement of the failure studied here: in a single model, window based pipeline, how much does suppressing alarms during an open window cost when a second change arrives, how much of that cost is due to window length, and how does it interact with replay? This is an evaluation question about a common engineering pattern, not an unsolved algorithmic problem.

3 Problem formulation

System and task. The state is $s_t = (p_t, v_t) \in \mathbb{R}^4$, with position p_t and velocity v_t in the plane, and the action is $a_t \in [-1, 1]^2$. The dynamics are governed by $\theta = (g, c, \varphi)$: actuator gain, linear drag, and actuator rotation. With $\Delta t = 0.1$,

$$v_{t+1} = v_t + \Delta t \left(g\, R(\varphi)\, a_t - c\, v_t \right) + \sigma \epsilon_t, \qquad p_{t+1} = p_t + \Delta t\, v_{t+1}, \tag{1}$$

where $R(\varphi)$ is the planar rotation matrix, $\epsilon_t \sim \mathcal{N}(0, I)$ and $\sigma = 0.02$. The arena is $[-2, 2]^2$; a position component that leaves it is clipped to the wall and the corresponding velocity component is set to zero. The defaults are $g = 1$, $c = 0.5$, $\varphi = 0$. Each episode draws a start and a goal p^* in $[-1.6, 1.6]^2$, at least 1 apart, and the reward is

$$r_t = -\left(\|p_{t+1} - p^*\| + 0.01\, \|a_t\|^2 \right). \tag{2}$$

An episode succeeds when $\|p_{t+1} - p^*\| < 0.15$ and otherwise ends after 100 steps. A successful episode typically returns about -25 to -35; a failed one returns roughly -80 to -250.

Non-stationarity. A run consists of 30 episodes. The dynamics parameters are piecewise constant over episodes and follow a schedule $\theta^{(0)}, \theta^{(1)}, \ldots$ that switches at the first step of given episodes. We call each constant piece a *regime* and the episodes it covers a *segment*. The agent is never told the schedule, the switch times or the parameter values.

Learned model and residual. The agent plans with a learned model $\bar{f}(s, a) = \frac{1}{E} \sum_{e=1}^{E} f_e(s, a)$, the mean of E ensemble members. After each step it scores the transition it has just observed with the model as it was *before* training on that transition,

$$e_t = \tfrac{1}{4} \left\| \bar{f}(s_t, a_t) - s_{t+1} \right\|^2. \tag{3}$$

Adaptation windows and the interruption problem. An alarm at step τ opens a window W that collects every later transition. The model is fine tuned when $|W|$ reaches $n_{\min}, n_{\min}+n_{\Delta}, n_{\min}+2n_{\Delta}, \ldots$ and the window closes after the fine tune at which $|W| \geq n_{\max}$. An *interruption* occurs when a regime change at step t_2 falls inside an open window, $\tau < t_2$ and the window has not closed. From t_2 on, W mixes transitions from two regimes. The question is what the agent should do with an alarm raised while W is open, given that it does not know whether the alarm reflects a new change or the residual error of incomplete adaptation.

4 Methods

World model and planner. The model is an ensemble of $E = 5$ multilayer perceptrons, each with two hidden layers of 128 SiLU units. Each member predicts the normalised state difference $s_{t+1} - s_t$, and members differ in initialisation and in a bootstrap resample of the training set. Two pieces of known structure are used: predictions pass through the simulator's wall rule, and transitions with a wall contact are excluded from training. In an earlier stage wall contacts made up 10.7% of test transitions but 85% of the squared error of a model without this rule (Appendix A). The model is pretrained on 30 episodes of temporally correlated random actions (60 epochs, Adam, learning rate 10^{-3}, weight decay 10^{-5}, batch 256). The planner is cross entropy model predictive control with horizon 15, 256 candidates, 25 elites, 4 iterations, initial standard deviation 0.6 and a warm start. Every member rolls out every candidate, the cost of a sequence is the member mean of its summed cost, and only the first action is executed.

Change detection. The main detector is a one sided CUSUM [18] on the standardised log residual,

$$z_t = \frac{\log(e_t + 10^{-10}) - \mu}{\varsigma}, \qquad S_t = \max(0,\, S_{t-1} + z_t - k), \qquad \text{alarm if } S_t > h, \tag{4}$$

with $k = 0.5$ and $h = 10$. The reference values μ and ς come from five calibration episodes run with the pretrained model on reset seeds never used for training. A detector based on ensemble disagreement was evaluated in earlier stages and rejected because it was slow and raised false alarms (Appendix A).

Fine tuning and replay. Windows use $n_{\min} = 50$, $n_{\Delta} = 100$ and $n_{\max} = 500$, giving six fine tunes, the last on 550 transitions. Each fine tune warm starts every member, runs 40 epochs of Adam at learning rate 10^{-3} on a bootstrap resample, keeps the normalisers of pretraining and excludes wall contacts. Detector statistics are reset after every fine tune. With replay ratio ρ, a fine tune on n window transitions adds

$$n_{\text{old}} = \text{round}\big(n\rho/(1-\rho)\big) \tag{5}$$

old transitions, drawn without replacement from the pretraining data together with all online transitions recorded before the current window started. With $\rho = 0.25$ one quarter of the fine tuning set is old data; with $\rho = 0$ ("fresh only") the set contains window data only.

Interruption policies. Figure 1 shows the state machine. An alarm in NORMAL opens a window and moves the agent to COLLECTING, where it stores transitions until the first fine tune; the agent is then in ADAPTING until the window closes. Main detector alarms are ignored in both window states. We compare three policies.

- ignore (the original behaviour): nothing else happens while a window is open.
- restart: a separate re-detection CUSUM on the same standardised residual runs only in ADAPTING, that is, after the window's first fine tune, and is reset after every fine tune. Its larger drift $k_r = 1.5$ (with $h_r = 10$) is needed because a partly adapted model keeps the residual moderately

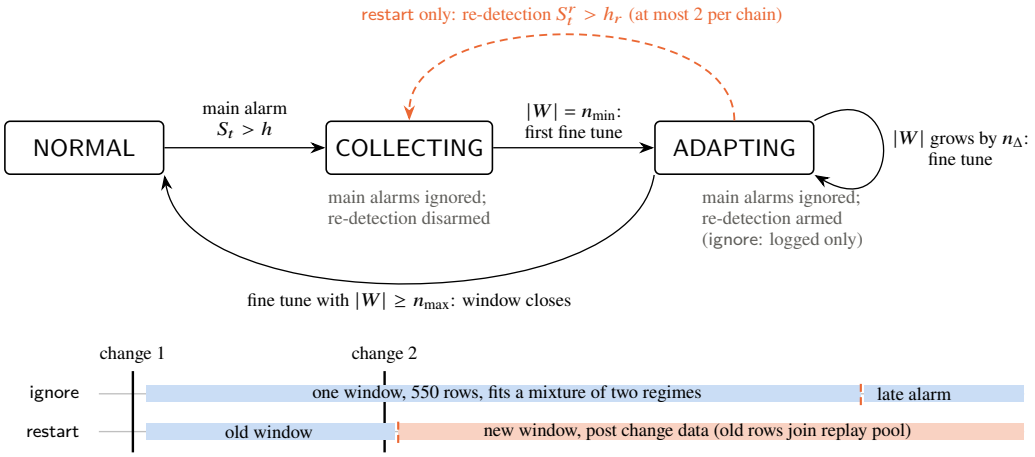


Figure 1: The adaptation state machine as implemented (`agent.py`). **Top:** states and transitions. Main alarms are always ignored while a window is open. Under restart a separate, stricter CUSUM ($k_r = 1.5$, $h_r = 10$) runs only in ADAPTING, is reset after each fine tune, and on firing closes the window and opens a new one. Under ignore the same statistic is computed but only logged. **Bottom:** a schematic (not to scale) of a second change inside an open window. Dashed lines are alarms; shaded bars are windows.

Algorithm 1 Per step update of the adaptation state machine

observe (s_t, a_t, s_{t+1}); compute e_t with the current model (Equation (3)); store the transition
if state = NORMAL **and** main CUSUM alarms **then**
    open window W at t; restarts $\leftarrow$ 0; state $\leftarrow$ COLLECTING
else if state = ADAPTING **and** re-detection CUSUM fires **then**
    if policy = restart **and** restarts < 2 **then**
        restarts $\leftarrow$ restarts+1; close W (its rows join the replay pool)
        open a new window W at t; state $\leftarrow$ COLLECTING
    else
        log the re-detection; reset the re-detection CUSUM
if a window is open **then**
    add the transition to W
    if $|W|$ reaches the next fine tune point ($n_{\min}$, then every n_Δ) **then**
        fine tune on W plus n_{old} replayed rows (Equation (5)); reset all detectors; state $\leftarrow$ ADAPTING
        if $|W| \geq n_{\max}$ **then** close W; state $\leftarrow$ NORMAL

high without any new change; in one logged run the main detector would have fired on 414 of 816 steps. When the re-detector fires and fewer than two restarts have occurred in the current chain of windows, the window is closed and a new one is opened at that step. The counter resets when a main alarm opens a new chain from NORMAL. The restart is a hard cut: transitions between the true change and the re-detection stay with the closed window and become replayable old data.

- ignore with a short window ("short"): $n_{\max} = 150$, so each window has two fine tunes. This is a competing explanation, not a proposed method. If restart helps only because it ends the stale window sooner, a short window should help as much. The value was fixed before any run, as the shortest window with more than one update.

Algorithm 1 summarises the per step logic. No policy uses the true schedule.

5 Experimental setup

Schedules. Every schedule has rotation 0° for episodes 0 to 7 and a first change to 60° at episode 8. They differ in what happens next (Table 1). The first change opens a window that, with $n_{\max} = 500$,

Table 1: Change schedules. All runs have 30 episodes; the first change is $0° \to 60°$ at episode 8.

Schedule	Second change	Episode	Used for tuning
`persistent`	none (control)	n/a	yes
`reversion`	rotation $60° \to 0°$	20	no
`second_change`	rotation $60° \to -30°$	11	yes
`late_second`	rotation $60° \to -30°$	15	no
`large_second`	rotation $60° \to -60°$	11	no
`gain_second`	gain $1 \to 0.5$, rotation stays $60°$	11	no

stays open for several episodes, so in every two change schedule the second change arrives while that window is still open under ignore and restart. The `persistent` schedule is the control: any restart there is false by construction. Three schedules were added after the restart policy had been tuned and were never used for any selection: a later second change, a larger one, and a change of a different kind.

Conditions. We cross two replay settings, fresh only ($\rho = 0$) and replay ($\rho = 0.25$), with the three policies, all with $n_{\min} = 50$. References are an agent that never adapts and, in the first three schedules, an oracle that is told the change episodes and opens a window at the first step of each. The oracle is a reference, not a method.

Seeds, tuning and evaluation. An agent seed fixes its pretraining data, model initialisation, calibration episodes, held out sets and the reset seeds of its 30 episodes, and the seed ranges for these are disjoint by construction. Within a seed all conditions start from the identical pretrained model and see identical episode resets, so comparisons are paired. The re-detection parameters were selected on tuning seeds 100 to 102 from the grid $k_r \in \{1.0, 1.5, 2.0\} \times h_r \in \{10, 20\}$ in the `persistent` and `second_change` schedules, by a rule fixed in advance: keep settings within one of the fewest false restarts, then take the highest `second_change` return, breaking ties towards the more conservative setting. The rule selected $k_r = 1.5$, $h_r = 10$, with one false restart in six control runs (Table 6). All results below use evaluation seeds 0 to 9, which were never used for selection. The replay ratio 0.25 and $n_{\min} = 50$ were selected earlier, by a similar rule on the same tuning seeds under a single permanent change.

Metrics. The main outcome is the *segment return*: the mean episode return over the episodes after the second change (after the only change in `persistent`). For replay we also report the whole run mean return. The *detection delay* is the number of steps from a change to the first alarm of any kind before the next change; under ignore that alarm can only come from the main detector after the window closes. A change with no alarm before the next change or the end of the run is counted as *not detected* and is reported separately, never dropped silently. *Episodes to recover* is the index, counted from the first episode after the change, of the first of three consecutive successful episodes. *Held out error* is the one step mean squared error on ten random policy episodes per regime, collected from reset seeds no agent trains on and used only for this measurement; we use it at the end of the run to measure forgetting. *Cost* is the number of fine tunes; wall clock times were measured with seven runs in parallel and are indicative only.

Statistics. The unit of analysis is the agent seed. For each comparison we report the mean of the per seed paired differences, their standard deviation, the number of seeds with a positive difference, and a 95% percentile bootstrap interval of the mean over seeds (10,000 resamples). With ten seeds and many comparisons these are descriptive summaries; we do not correct for multiplicity and do not claim significance. Agreement in sign across all ten seeds is the most robust evidence we have.

Table 2: Return after the second change (after the only change in `persistent`): paired differences over ten evaluation seeds. Mean, standard deviation (SD), 95% bootstrap interval of the mean (CI) and the number of seeds with a positive difference (k). Returns are negative, so a positive difference favours the first policy named. "short" is ignore with $n_{\max} = 150$.

		restart − ignore				restart − short			
Schedule	Replay	Mean	SD	CI	k	Mean	SD	CI	k
persistent	fresh only	−0.01	0.03	[−0.02, 0.00]	0/10	0.02	0.06	[−0.02, 0.05]	6/10
	replay 0.25	−0.03	0.10	[−0.10, 0.02]	1/10	0.24	0.60	[−0.10, 0.62]	8/10
reversion	fresh only	4.93	1.49	[4.07, 5.85]	10/10	−0.68	1.02	[−1.24, −0.07]	2/10
	replay 0.25	1.60	1.83	[0.62, 2.77]	9/10	−0.04	0.76	[−0.45, 0.43]	5/10
second_change	fresh only	6.83	3.83	[4.73, 9.19]	10/10	2.51	2.34	[1.23, 3.92]	10/10
	replay 0.25	2.41	1.34	[1.65, 3.22]	10/10	0.34	1.01	[−0.33, 0.85]	8/10
late_second	fresh only	13.53	3.39	[11.56, 15.46]	10/10	−0.62	1.02	[−1.25, −0.09]	3/10
	replay 0.25	5.39	2.19	[4.04, 6.64]	10/10	1.14	2.80	[−0.42, 2.81]	5/10
large_second	fresh only	27.16	10.45	[21.80, 33.84]	10/10	12.44	8.13	[7.82, 17.26]	10/10
	replay 0.25	12.57	8.85	[8.16, 18.32]	10/10	3.84	4.14	[1.35, 6.19]	8/10
gain_second	fresh only	0.03	0.07	[−0.01, 0.07]	3/10	−0.12	0.26	[−0.29, 0.02]	3/10
	replay 0.25	−0.27	3.67	[−2.49, 1.76]	5/10	−1.58	5.47	[−4.51, 1.88]	2/10

6 Results

6.1 RQ1: restart against ignore

Table 2 and Figure 2 give the central result. In every schedule where the second change is a rotation, restart improved the return after it in all ten seeds without replay, and in 39 of 40 seed comparisons with replay. The improvement grows with the size of the second change: +6.83 for a −30° change at episode 11 and +27.16 for a −60° change at the same episode, without replay. For reference, the difference between a successful and a failed episode is roughly 50 to 200 return units, so these gains reflect episodes that fail under ignore and succeed under restart (Figure 8; absolute returns are in Table 7). The gain is smaller with replay in all four rotation schedules, for reasons we return to in Section 6.3. In the `persistent` control, restart changed almost nothing (−0.01 and −0.03), and after the gain change it made no measurable difference.

6.2 The competing explanation: a shorter window

If restart helps mainly because the stale window ends sooner, ignore with a short window should recover the same gain. Figure 3 plots, for each seed, the short window's improvement over ignore against restart's. Points on the diagonal mean the short window did as well as restart; points on the dashed line mean it recovered half of restart's gain. Measured by the ratio of means, the short window recovered 114% of restart's gain in `reversion`, 63% in `second_change`, 105% in `late_second` and 54% in `large_second` without replay, and between 69% and 102% with replay.

The right half of Table 2 compares the two directly. Restart was clearly better than the short window in two places: `second_change` without replay (+2.51, 10 of 10 seeds) and `large_second` (+12.44 without replay, 10 of 10; +3.84 with replay, 8 of 10). It was slightly *worse* after a reversion without replay (−0.68, 2 of 10 seeds positive, interval [−1.24, −0.07]) and in `late_second` without replay (−0.62, 3 of 10). Elsewhere the two were indistinguishable at ten seeds. The competing explanation is therefore partly right: much of the cost of the original policy comes from the length of the window rather than from suppressing the alarm itself.

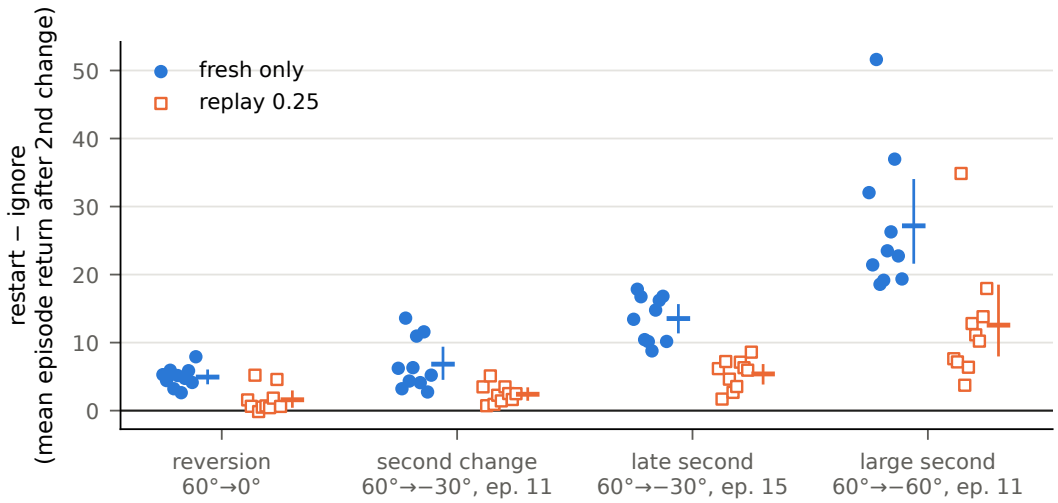


Figure 2: Restart minus ignore, mean episode return after the second change, for the four rotation schedules. Each marker is one evaluation seed (filled circles: fresh only; open squares: replay 0.25). The horizontal tick is the mean over ten seeds and the vertical line its 95% bootstrap interval. All 40 fresh only differences are positive, and 39 of 40 with replay.

Table 3: Whole run mean return, replay 0.25 minus fresh only, under each policy: mean, 95% bootstrap interval and number of positive seeds out of ten.

	under ignore			under short			under restart		
Schedule	Mean	CI	k	Mean	CI	k	Mean	CI	k
persistent	−0.55	[−0.65, −0.46]	0/10	−0.73	[−0.95, −0.57]	0/10	−0.57	[−0.70, −0.45]	0/10
reversion	1.96	[1.54, 2.48]	10/10	0.60	[0.26, 0.94]	8/10	0.84	[0.62, 1.12]	10/10
second_change	3.31	[2.32, 4.41]	10/10	1.87	[1.10, 2.72]	10/10	0.51	[−0.22, 1.04]	9/10
late_second	5.28	[4.08, 6.58]	10/10	0.33	[−0.52, 1.18]	6/10	1.23	[0.83, 1.68]	10/10
large_second	10.27	[7.80, 13.11]	10/10	6.48	[3.79, 9.04]	10/10	1.03	[−0.23, 2.35]	6/10
gain_second	−2.97	[−3.87, −2.18]	0/10	−2.24	[−3.76, −1.30]	0/10	−3.16	[−4.57, −1.99]	0/10

6.3 RQ3 and RQ4: replay depends on the interruption policy

Replay looks very different depending on the policy it is paired with (Table 3, Figure 4). Under ignore, replay improved whole run return in every seed of every rotation two change schedule, by up to +10.27 on average in `large_second`. Under restart, most of that benefit disappeared: +0.84 in `reversion` and +1.23 in `late_second`, both positive in all ten seeds, while the intervals for `second_change` and `large_second` include zero. The short window sits in between. The `large_second` result is informative: there, the new regime (−60°) is as far from the pretraining regime (0°) as the intermediate one (60°), yet replay still added +10.27 under ignore. After a single permanent change and after the gain change, replay hurt: by about −0.55 to −0.73 in `persistent` and −2.2 to −3.2 after the gain change, negative in every seed under every policy (Figure 7b).

For RQ4 we combine the earlier sweep under a single permanent change (seeds 0 to 4, fresh only versus replay, Table 5) with the reversion results. After a permanent change, ratio 0 gave the best return and the cost of replay grew with the size of the change: at 90° the post change return was −29.80, −32.62 and −37.94 for ratios 0, 0.25 and 0.5. After a reversion, ratio 0.25 was better under both policies (return after reverting −30.17 against −37.02 under ignore, −28.57 against −32.09 under restart). No single ratio was best for both kinds of change.



Figure 3: How much of restart's improvement a shorter window recovers. Each small marker is one seed: the horizontal axis is restart minus ignore and the vertical axis is short window minus ignore, both in mean episode return after the second change. Large filled markers are means; the legend gives the ratio of the two means. Solid line: the short window matches restart. Dashed line: it recovers half of restart's gain.

6.4 Detection and recovery

Table 4 and Figure 5 show the mechanism behind Table 2. Under ignore, the first alarm after a second change came 236 to 484 steps late on average, several episodes, because the main detector cannot act until the first window closes. Under restart, the re-detection fired 5 to 11 steps after a rotation change on average; the exception is the reversion with replay (40.8 steps), where replay had kept the model closer to 0° and the residual rose less. The short window fell in between: about 35 to 40 steps in `second_change` and `large_second`, where the change hit a window that was still open, and 2.5 steps in `late_second` without replay, where the short window had already closed and the main detector was armed. With replay under ignore, the reversion went undetected in 6 of 10 seeds: the held out error on 0° just before the reversion was 9.3×10^{-4} with replay against 1.6×10^{-3} fresh only, so the residual barely rose. Recovery times follow the same order. In `large_second` without replay, the agent took 3.1 episodes to recover under ignore, 1.7 with the short window and 0.6 under restart. Figure 9 in the appendix shows one seed in detail.

6.5 RQ2: the costs of restart

False restarts. In the `persistent` control every restart is false. Without replay there was one false restart in ten runs (seed 5), and with replay four, in seeds 0 (two), 2 and 5. Neither changed return in the control measurably (Table 2). The rate matches the tuning sweep, which showed one false restart in six control runs. Seed 5's false restart occurs before episode 11 and therefore appears in every schedule, since all schedules are identical up to that point; it is one event observed six times.

Follow up restarts and compute. After a large second change, the new window's first fine tune, on only 50 transitions, often left the residual high enough to trigger a second restart. This happened in 9 of 10 seeds in `large_second` under both replay settings and in `late_second` without replay, using up the restart budget. Restart nonetheless used about as many or fewer fine tunes than ignore, because each restart replaces the remaining fine tunes of the old window (Table 8 in the appendix); the short window was the cheapest policy.

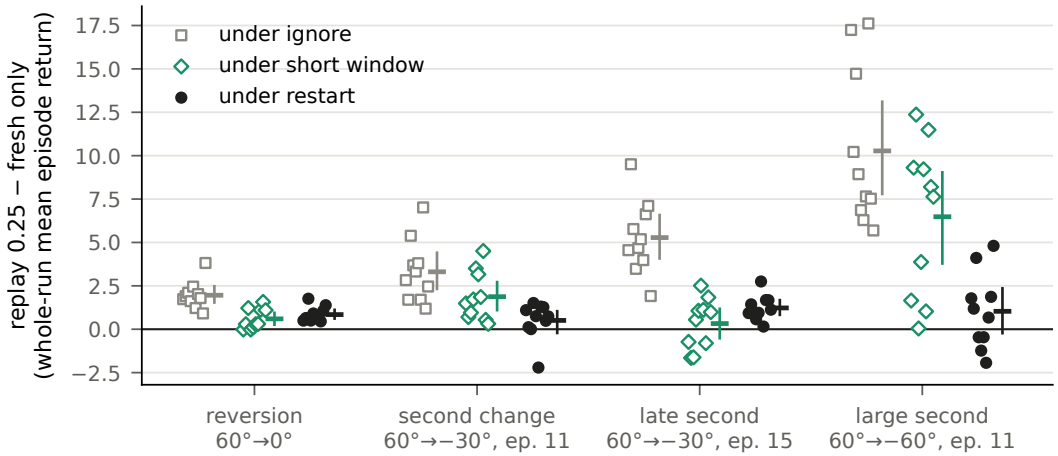


Figure 4: Replay 0.25 minus fresh only, whole run mean episode return, under each interruption policy. One marker per seed; ticks and vertical lines are means and 95% bootstrap intervals. Replay's benefit is large under ignore and small under restart.

Table 4: Detection of the second change and recovery afterwards. "Delay": mean steps from the change to the first alarm, over seeds in which an alarm occurred; a superscript (n) counts seeds with no alarm before the next change or the end of the run. "Rec.": mean episodes to recover (index of the first of three consecutive successes after the change; 0 means immediately).

		ignore		short		restart	
Schedule	Replay	Delay	Rec.	Delay	Rec.	Delay	Rec.
reversion	fresh only	245.8	0.5	20.9	0.0	8.4	0.0
	replay 0.25	$235.8^{(6)}$	0.0	$15.7^{(1)}$	0.0	$40.8^{(1)}$	0.0
second_change	fresh only	473.2	0.2	39.2	0.2	$6.7^{(1)}$	0.0
	replay 0.25	$473.1^{(2)}$	0.1	37.4	0.0	8.0	0.0
late_second	fresh only	358.6	1.4	2.5	0.0	5.6	0.0
	replay 0.25	355.5	0.4	98.6	0.4	10.5	0.0
large_second	fresh only	449.1	3.1	39.4	1.7	9.6	0.6
	replay 0.25	446.7	1.6	35.4	1.3	5.0	0.9
gain_second	fresh only	$483.6^{(5)}$	0.0	56.5	0.0	$155.9^{(3)}$	0.0
	replay 0.25	467.9	0.0	50.1	0.0	209.9	0.0

Forgetting. Restart pays in the intermediate regime (Figure 6). After a reversion, restart and the short window both ended with a worse model of 60° than ignore did. The stale window under ignore keeps 60° transitions in the fine tuning set until it closes, which preserves the intermediate regime slightly better; this is the same mixing that hurts control after the change. Replay reduced forgetting of 60° under every policy and schedule shown, at the cost of a less accurate model of the newest regime: on −60°, for example, restart reached 5.6×10^{-4} with replay against 2.6×10^{-4} without.

6.6 Where restart does not help

Figure 7 collects the settings where restart had no benefit. In the `persistent` control the fresh only difference was exactly zero in nine seeds, because no restart fired. The halved gain in `gain_second` was barely visible to the agent. The first episode after it returned −33.6 in every adaptive condition, no worse than before the change, and success after the change was 99 to 100%. The residual rose only slightly: restart without replay missed the change in 3 of 10 seeds and took 156 steps on average otherwise. Neither restart nor the short window changed return measurably, while replay lowered it in every seed under every policy.

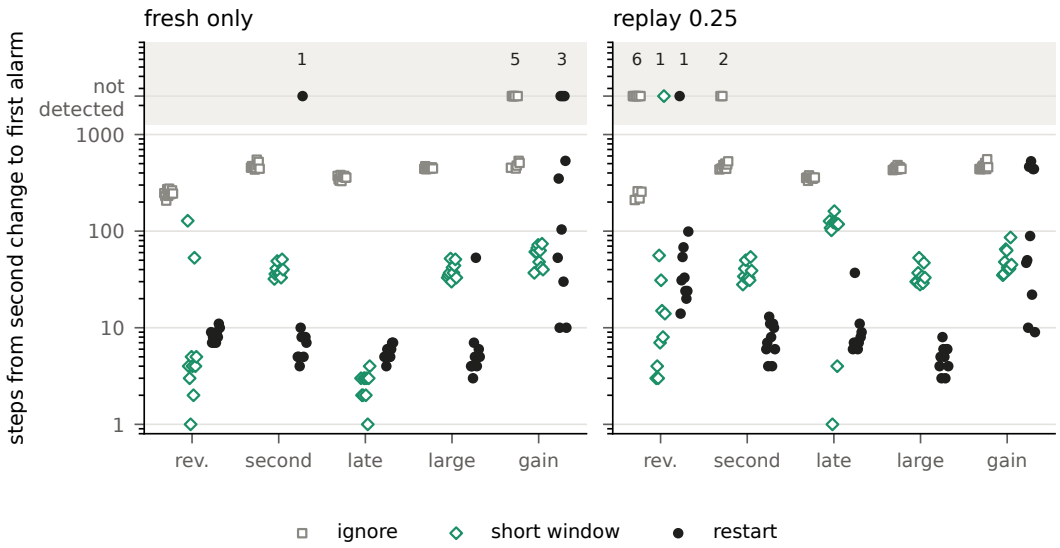


Figure 5: Steps from the second change to the first alarm of any kind, per seed, on a log scale. Markers in the shaded band were not detected before the next change or the end of the run; the number above the band counts them. Schedules: reversion, second change, late second, large second and gain second. The ignore policy is blind for hundreds of steps; restart reacts within about ten steps to rotation changes but responds weakly to the gain change.

7 Discussion

Why ignore fails. The observations above separate two failure modes of the original policy. The first is blindness: the agent cannot respond to a second change for as long as the window stays open, which here meant several hundred steps. The second is mixing: every fine tune in the stale window fits a single model to transitions from two regimes. Shortening the window addresses blindness directly and mixing partially, because fewer fine tunes see mixed data. Restart addresses both, at the cost of a second detector with its own false alarms.

Why a short window recovers so much. In `late_second` the second change arrived after the short window had already closed, so the main detector caught it within a few steps and the short window behaved like an ideal restart. After a reversion to the pretraining regime the model needed little adaptation, and two fine tunes sufficed. Restart kept its advantage when a large change hit a window that was still open even at $n_{\max} = 150$: the short window then detected the change only after closing, about 39 steps late, adapted with two fine tunes, and had to wait for the main detector to open another window, which slowed recovery. This account is consistent with the delays and recovery times in Table 4, but we have not tested it directly. We did not find a mechanism for the small reversion deficit of restart relative to the short window. The model error on 0° just before the reversion was the same under both (1.6 to 1.7×10^{-3}), and restart detected the reversion earlier (8.4 against 20.9 steps). We report this difference as unexplained.

Replay as a partial substitute for interruption handling. Replay helped most where ignore hurt most. A plausible reading is that, with a quarter of every fine tuning batch drawn from earlier data, each fine tune in a stale window is pulled less towards the regime that has just ended, which softens the mixing problem. Once restart cuts the stale window, little of that benefit remains. The `large_second` result argues against the simplest alternative, that replay helps only because the new regime is closer to pretraining. Under restart, the remaining benefit was clearest after a genuine reversion and in `late_second`, where −30° is closer to 0° than 60° is. Ten seeds do not separate these two explanations further. The practical consequence is that a study comparing replay ratios under an

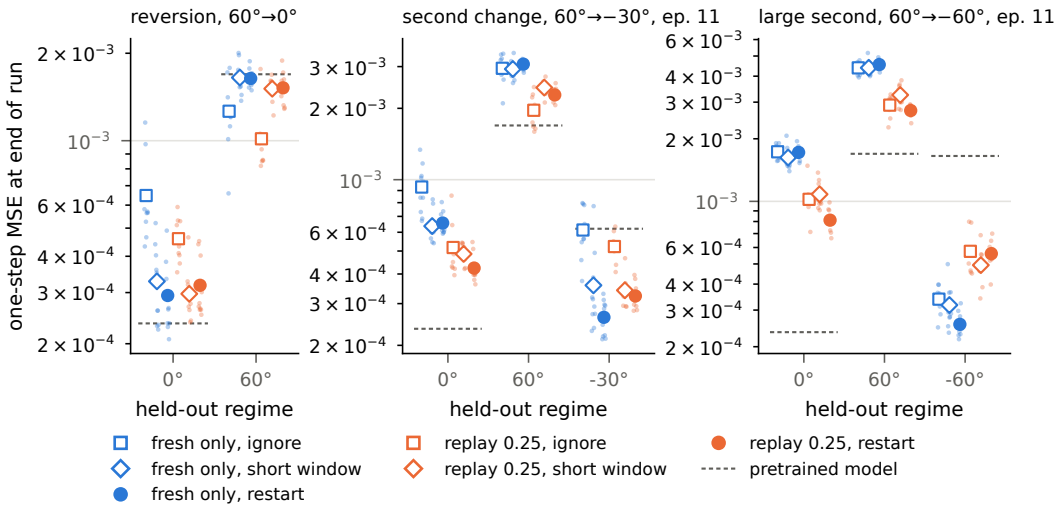


Figure 6: Held out one step mean squared error at the end of the run, for each regime the agent has seen. Large markers are means over ten seeds and small dots are individual seeds; the dashed line is the pretrained model. Restart and the short window forget the intermediate 60° regime more after a reversion; replay retains it better but fits the newest regime less well.

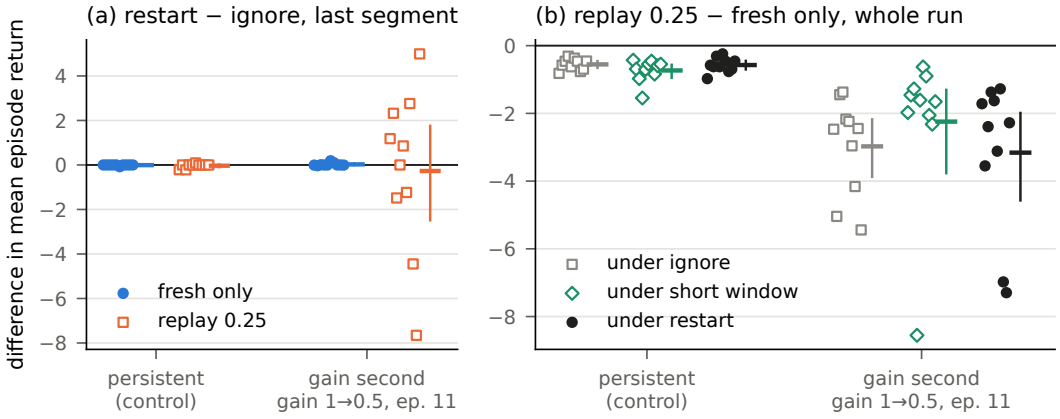


Figure 7: Negative and null results. (a) Restart minus ignore in the single change control and after the gain change: no measurable effect. (b) Replay 0.25 minus fresh only, whole run return: negative in every seed under every policy in both settings. One marker per seed; ticks and lines are means and 95% bootstrap intervals.

alarm suppressing policy would credit replay with a benefit that mostly comes from compensating for the policy. Here, the replay ratio and the interruption policy have to be chosen together. After the gain change, replay was harmful under every policy. A likely cause, which we did not test, is that the replayed old data include full gain transitions at 60° that now misrepresent the dynamics.

Detector limitations and an edge case. The re-detector was tuned on rotation changes and responded weakly to the gain change. That change also mattered little for control, so the gain schedule tests specificity more than sensitivity. The delay metric has one edge case worth stating. In `second_change` without replay, seed 5's false re-detection fired 5 steps before the true change; the new window's first fine tune absorbed the change and the agent recovered (restart minus ignore was +10.96 in this seed), but because no alarm followed the change, it counts as not detected. The one undetected reversion under replay with restart is similar: a false restart opened a window 37 steps before the reversion, and that window's fine tunes absorbed it.

A combined policy. Restart and a short window address different parts of the problem and could be combined, with a short window removing the long blind period and re-detection catching a change inside the remaining window. We did not run this combination. Choosing it after seeing the evaluation seeds would amount to selecting a method on the evaluation data, and a fair test needs fresh tuning and evaluation seeds.

8 Limitations

All results come from one small simulated system: a four dimensional point mass with a known wall rule and Gaussian noise. Actuator rotation is a convenient, strong change; many real changes are subtler or gradual, and nothing here shows that the effect sizes carry over to higher dimensional systems, image observations or physical robots. Each schedule has at most two changes at fixed episodes; changes within an episode, gradual drift and long change sequences were not studied.

With ten evaluation seeds per condition, our statistics are descriptive and uncorrected for the many comparisons, and the intervals of the smaller effects include zero. The detectors were tuned on rotation changes, which partly explains the weak response to the gain change. The held out error uses random policy data, so it measures the model on a different state distribution from the one the planner visits.

We did not compare against per context model libraries such as MBCD [2], MOLe [16] or Gaussian process mixtures [22]. They are the natural stronger baselines, particularly for the reversion schedule, and our results say nothing about how a single model with restart compares to them. Of the possible alternatives to restart, only a shorter window was evaluated; the combined restart and short window policy, sliding windows and local forgetting [19] remain untested. Finally, wall clock costs were measured on a shared laptop CPU and are only indicative.

9 Conclusion

In a single model, window based adaptation pipeline, suppressing alarms while an adaptation window is open was costly when a second rotation change arrived inside the window. Restarting the window on a separate, stricter re-detection improved return after that change in every seed of every rotation schedule without replay, and in 39 of 40 seed comparisons with replay. It had no measurable cost in the single change control, but it forgot the intermediate regime somewhat more (RQ1, RQ2). A large part of this improvement came from ending the stale window sooner rather than from acting on the alarm: a plain ignore policy with a shorter window recovered at least half of it everywhere and all of it in two of four rotation schedules. Restart kept a clear advantage only when a large change landed in a still open window. Replay with ratio 0.25 helped strongly under the original policy, mainly because it compensated for stale windows, helped little under restart, and reduced return after a single permanent change and after the gain change (RQ3). No single replay ratio was best for both permanent and reversible changes (RQ4).

For practitioners using such a pipeline, the narrow lesson these data support is to choose the interruption policy, the window length and the replay ratio together, and to avoid a long adaptation window with suppressed alarms. Whether this holds in richer systems, and how a single model with restart compares with a library of per context models, are open questions that this study does not answer.

References

[1] R. P. Adams and D. J. C. MacKay. Bayesian online changepoint detection, 2007.

[2] L. N. Alegre, A. L. C. Bazzan, and B. C. da Silva. Minimum-delay adaptation in non-stationary

reinforcement learning via online high-confidence change-point detection. In *Proceedings of the 20th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, pages 97–105, 2021.

[3] A. Bifet and R. Gavaldà. Learning from time-changing data with adaptive windowing. In *Proceedings of the 2007 SIAM International Conference on Data Mining (SDM)*, pages 443–448, 2007. doi: 10.1137/1.9781611972771.42.

[4] K. Chua, R. Calandra, R. McAllister, and S. Levine. Deep reinforcement learning in a handful of trials using probabilistic dynamics models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

[5] B. C. da Silva, E. W. Basso, A. L. C. Bazzan, and P. M. Engel. Dealing with non-stationary environments using context detection. In *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, pages 217–224, 2006. doi: 10.1145/1143844.1143872.

[6] H. Fu, Y. Sun, M. Littman, and G. Konidaris. Knowledge retention in continual model-based reinforcement learning. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *Proceedings of Machine Learning Research*, pages 17832–17851, 2025. URL https://proceedings.mlr.press/v267/fu25f.html.

[7] J. Gama, P. Medas, G. Castillo, and P. Rodrigues. Learning with drift detection. In *Advances in Artificial Intelligence, SBIA 2004*, volume 3171 of *Lecture Notes in Computer Science*, pages 286–295, 2004. doi: 10.1007/978-3-540-28645-5_29.

[8] Y. Huang, K. Xie, H. Bharadhwaj, and F. Shkurti. Continual model-based reinforcement learning with hypernetworks. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2021.

[9] K. Khetarpal, M. Riemer, I. Rish, and D. Precup. Towards continual reinforcement learning: A review and perspectives. *Journal of Artificial Intelligence Research*, 75:1401–1476, 2022. doi: 10.1613/jair.1.13673.

[10] J. Kirkpatrick et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017. doi: 10.1073/pnas.1611835114.

[11] B. Lakshminarayanan, A. Pritzel, and C. Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[12] K. Lee, Y. Seo, S. Lee, H. Lee, and J. Shin. Context-aware dynamics model for generalization in model-based reinforcement learning. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020.

[13] Z. Liu, J. Lu, J. Xuan, and G. Zhang. Deep reinforcement learning in nonstationary environments with unknown change points. *IEEE Transactions on Cybernetics*, 54(9):5191–5204, 2024. doi: 10.1109/TCYB.2024.3356981.

[14] Z. Liu, J. Lu, J. Xuan, and G. Zhang. Learning latent and changing dynamics in real non-stationary environments. *IEEE Transactions on Knowledge and Data Engineering*, 37(4):1930–1942, 2025. doi: 10.1109/TKDE.2025.3535961.

[15] A. Nagabandi, I. Clavera, S. Liu, R. S. Fearing, P. Abbeel, S. Levine, and C. Finn. Learning to adapt in dynamic, real-world environments through meta-reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2019.

[16] A. Nagabandi, C. Finn, and S. Levine. Deep online learning via meta-learning: Continual adaptation for model-based RL. In *International Conference on Learning Representations (ICLR)*, 2019.

[17] S. Padakandla, Prabuchandran K. J., and S. Bhatnagar. Reinforcement learning algorithm for non-stationary environments. *Applied Intelligence*, 50(11):3590–3606, 2020. doi: 10.1007/s10489-020-01758-5.

[18] E. S. Page. Continuous inspection schemes. *Biometrika*, 41(1–2):100–115, 1954. doi: 10.1093/biomet/41.1-2.100.

[19] A. Rahimi-Kalahroudi, J. Rajendran, I. Momennejad, H. van Seijen, and S. Chandar. Replay buffer with local forgetting for adapting to local environment changes in deep model-based reinforcement learning. In *Proceedings of the 2nd Conference on Lifelong Learning Agents (CoLLAs)*, volume 232 of *Proceedings of Machine Learning Research*, pages 21–42, 2023. URL https://proceedings.mlr.press/v232/rahimi-kalahroudi23a.html.

[20] D. Rolnick, A. Ahuja, J. Schwarz, T. P. Lillicrap, and G. Wayne. Experience replay for continual learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[21] H. van Seijen, H. Nekoei, E. Racah, and S. Chandar. The LoCA regret: A consistent metric to evaluate

model-based behavior in reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[22] M. Xu, W. Ding, J. Zhu, Z. Liu, B. Chen, and D. Zhao. Task-agnostic online reinforcement learning with an infinite mixture of Gaussian processes. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

A Evidence from earlier stages

The design choices in Section 4 rest on earlier stages of the same project. Their numbers were recomputed from the saved results during an audit before this study. A model with the wall rule had one step error 1.40×10^{-3}, against 2.83×10^{-3} for a constant velocity predictor and 3.58×10^{-3} for predicting no change. Without the rule, wall contacts (213 of 2000 test transitions) carried 85% of the squared error. Planning with the learned model returned −31.8 against −31.1 for a tuned proportional derivative controller, both with 100% success, at 26.0 ms against 0.0032 ms per decision. Ensemble disagreement correlated with error (Spearman 0.461 on stationary data, 0.292 after a gain and drag change), but a detector built on it took about 405 steps and raised false alarms, so the residual detector was used. After a 60° or 90° rotation, a non adapting agent succeeded in no episode; with residual detection and adaptation it succeeded in 95.5% of post change episodes, with detection delays of 3.6 and 2.6 steps.

B Replay ratio under a single permanent change

Table 5: Earlier sweep (seeds 0 to 4, $n_{\min} = 50$, fresh only versus replay): return after a single permanent rotation and success in the first episode after the change.

	Post change return			First episode success		
Rotation	$\rho = 0$	0.25	0.5	$\rho = 0$	0.25	0.5
60°	−27.25	−28.01	−28.88	5/5	5/5	5/5
90°	−29.80	−32.62	−37.94	5/5	3/5	1/5

C Re-detection tuning

Table 6: Re-detection grid on tuning seeds 100 to 102 (`persistent` and `second_change`, fresh only and replay; six control runs). The `second_change` return is the mean over tuning runs. The rule kept settings within one of the fewest false restarts (zero), then chose the highest return.

k_r	h_r	False restarts	`second_change` return	
1.0	10	9	−32.95	
1.0	20	6	−31.86	
1.5	10	1	−31.61	selected
1.5	20	0	−32.08	
2.0	10	0	−32.13	
2.0	20	0	−33.89	

D Reproducibility

The code, configurations, per seed results and logs are in the repository linked on the first page. The agent and state machine are in `src/autonomous_agent/agent.py`, the experiment runner in `scripts/run_redetect.py`, and the analysis in `scripts/analyze_research.py`. The configuration `configs/redetect60_ext.yaml` has settings identical to the earlier `configs/redetect60.yaml`, plus the short window conditions and the three new schedules. Results for seeds 0 to 4 in the first three schedules come from `results/s7_redetect`, and the remaining runs from `results/s8_short_s0_4`, `results/s8_seeds5_9` and `results/s8_newscen`.

Each run directory stores the command that produced it. The 375 runs added for this study took 6648 seconds of wall clock time on a laptop CPU with seven workers, with Python 3.13.7, PyTorch 2.14.1 and NumPy 2.5.4. Runs are deterministic on one machine for a given seed; two cross checks reproduced earlier results exactly, and we have not checked bitwise reproducibility across machines. Every figure and table in this paper is generated from the saved results by `paper/make_figures.py`.

E Additional results

The tables and figures below give absolute returns, compute, the per episode return curves and one run in detail. All are generated from the same saved results as the main text.

Table 7: Absolute return after the second change (after the only change in `persistent`): mean ± standard deviation over ten seeds. "None" never adapts.

		fresh only			replay 0.25		
Schedule	None	ignore	short	restart	ignore	short	restart
persistent	−94.8 ± 9.9	−29.3 ± 3.3	−29.3 ± 3.3	−29.3 ± 3.3	−30.1 ± 3.3	−30.3 ± 3.2	−30.1 ± 3.3
reversion	−26.9 ± 5.2	−37.0 ± 6.6	−31.4 ± 6.0	−32.1 ± 5.8	−30.2 ± 6.0	−28.5 ± 5.1	−28.6 ± 5.4
second_change	−29.4 ± 4.0	−37.7 ± 6.6	−33.3 ± 4.8	−30.8 ± 3.8	−32.3 ± 4.9	−30.3 ± 4.3	−29.9 ± 4.1
late_second	−29.2 ± 4.2	−47.9 ± 5.5	−33.7 ± 4.0	−34.3 ± 4.3	−36.9 ± 5.4	−32.7 ± 4.1	−31.5 ± 3.9
large_second	−95.9 ± 6.9	−66.8 ± 15.4	−52.0 ± 13.4	−39.6 ± 6.8	−50.4 ± 12.4	−41.7 ± 7.9	−37.9 ± 4.7
gain_second	−113.4 ± 11.5	−41.5 ± 5.6	−41.4 ± 5.5	−41.5 ± 5.6	−46.1 ± 6.3	−44.8 ± 4.4	−46.4 ± 8.0

Table 8: Mean number of fine tunes per run and total restarts in ten runs under restart (in `persistent` every restart is false).

		Fine tunes per run			Restarts
Schedule	Replay	ignore	short	restart	(10 runs)
persistent	fresh only	5.4	2.4	5.4	1
	replay 0.25	5.6	5.5	5.7	4
reversion	fresh only	6.9	4.5	5.9	12
	replay 0.25	6.0	5.8	5.6	12
second_change	fresh only	6.9	5.1	5.9	13
	replay 0.25	6.3	5.1	5.9	11
late_second	fresh only	7.8	4.9	6.2	20
	replay 0.25	7.0	6.5	6.1	15
large_second	fresh only	9.1	6.4	6.4	20
	replay 0.25	8.2	7.7	6.6	20
gain_second	fresh only	6.7	4.8	6.8	7
	replay 0.25	8.3	6.8	8.2	12

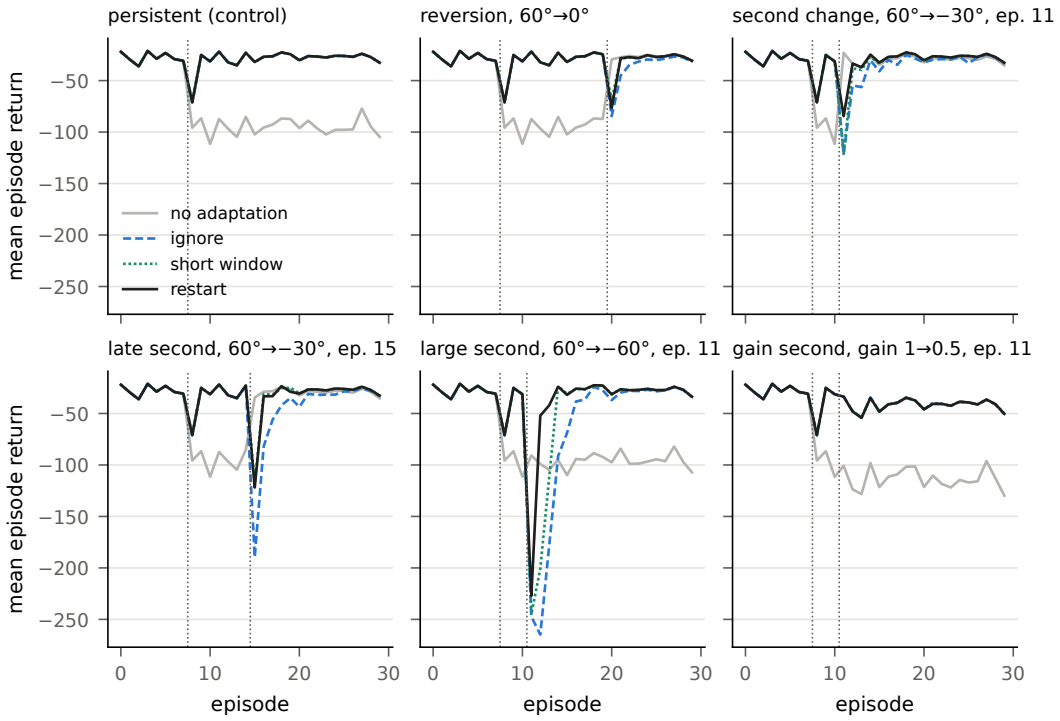


Figure 8: Mean episode return per episode over ten seeds, fresh only, unsmoothed. Dotted lines mark changes. The deep dips are failed episodes immediately after a change; their depth and width show how long each policy takes to recover.

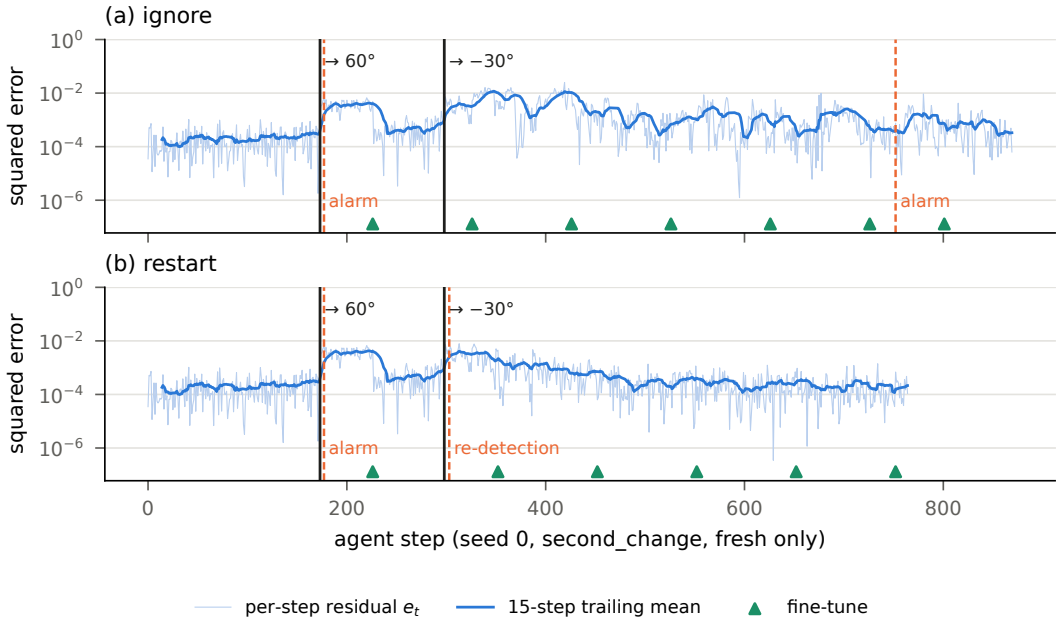


Figure 9: One run in detail (seed 0, `second_change`, fresh only). Light line: per step residual e_t; dark line: its 15 step trailing mean; black lines: true changes; dashed lines: alarms; triangles: fine tunes. (a) Under ignore the residual stays high for hundreds of steps after the change at step 298 while the window keeps fine tuning on mixed data; the next alarm comes at step 752. (b) Under restart the re-detection fires at step 303 and opens a new window.